

Project 1: Relational Database Design Using Generative AI

CSCI 362 Database Management Systems

Luaniris Garcia Diaz

Jay-Mart Database

May 11, 2026

PART I

Using Claude AI

First Input:

“Jay-Mart is a small (fictional) multi-department retail store with approximately 50 employees, several product departments (e.g., Electronics, Clothing, Grocery), and a growing customer base. I need you to design a database to support day-to-day operations, including employee management, inventory tracking, customer records, and sales transactions. First, describe the corporate and operational structure of a small department store. Then, identify entity types the business manages, data stored per entity, primary and foreign key candidates, and relevant business rules or constraints.”

Output:

Jay-Mart: Corporate & Operational Structure

A small department store like Jay-Mart typically organizes around two interlocking layers: a **corporate structure** (who reports to whom) and an **operational structure** (how the store actually runs day to day).

Corporate Structure

At the top sits a **Store Manager** who oversees the entire operation. Beneath them are two functional branches:

Administrative branch handles the back-office: HR (hiring, payroll, scheduling), accounting (vendor payments, revenue reporting), and loss prevention. These roles rarely touch the sales floor but their data — employee records, pay rates, audit logs — underpins everything else.

Operations branch is organized by department. Each department (Electronics, Clothing, Grocery, etc.) has a **Department Manager** who supervises a team of **Sales Associates** and, where relevant, **Stock Associates**. A **Shift Supervisor** may float across departments to cover managerial gaps without holding a permanent department role.

Operational Structure

Day-to-day store operations decompose into four interlocking workflows:

Inventory flow — Goods arrive from suppliers via purchase orders, are received and logged into stock, shelved in a department, and drawn down as items sell. Reorder points trigger new purchase orders when stock falls too low.

Sales flow — A customer selects items, a transaction is opened at a register (tied to the cashier employee), line items are recorded, payment is processed, and inventory is decremented. Returns reverse portions of this flow.

Employee flow — Employees are hired into a department and role, scheduled for shifts, and tracked for attendance. Pay is calculated from hours worked against their pay rate.

Customer flow — Customers may be anonymous (cash, no record) or registered (loyalty program). Registered customers accumulate a purchase history, enabling targeted promotions and return verification.

Entity Analysis

1. DEPARTMENT

The store's top-level organizational and merchandise unit.

Attribute	Notes
<code>department_id</code> (PK)	Surrogate key
<code>department_name</code>	e.g., Electronics, Clothing, Grocery
<code>floor_location</code>	Aisle/zone reference
<code>manager_id</code> (FK → EMPLOYEE)	Nullable; set after hire

Business rules: A department must exist before employees or products are assigned to it. The `manager_id` creates a self-referencing constraint resolved through the EMPLOYEE table.

2. EMPLOYEE

Every person on Jay-Mart's payroll, from the store manager to part-time stock associates.

Attribute	Notes
<code>employee_id</code> (PK)	Surrogate key
<code>first_name, last_name</code>	
<code>ssn</code>	Unique, encrypted at rest
<code>date_of_birth</code>	For age verification (e.g., alcohol section)
<code>hire_date</code>	
<code>termination_date</code>	Null = currently active
<code>role</code>	Enum: Store Manager, Dept Manager, Supervisor, Sales Associate, Stock Associate, Cashier

<code>pay_rate</code>	Hourly or salary
<code>pay_type</code>	Enum: Hourly, Salary
<code>department_id</code> (FK → DEPARTMENT)	Nullable for store-level roles
<code>manager_id</code> (FK → EMPLOYEE)	Self-referencing; direct supervisor

Business rules: An employee cannot be their own manager. A department manager must belong to the department they manage. Terminated employees (`termination_date IS NOT NULL`) should not appear on active schedules.

3. SHIFT

Scheduled and actual work periods for hourly employees.

Attribute	Notes
<code>shift_id</code> (PK)	
<code>employee_id</code> (FK → EMPLOYEE)	
<code>scheduled_start</code>	Datetime
<code>scheduled_end</code>	Datetime
<code>actual_start</code>	Null until clock-in
<code>actual_end</code>	Null until clock-out
<code>department_id</code> (FK → DEPARTMENT)	Where they worked that shift

Business rules: `scheduled_end` must be after `scheduled_start`. An employee cannot have two overlapping shifts. Hours worked = `actual_end - actual_start`, used in payroll

4. SUPPLIER

Vendors who supply merchandise to Jay-Mart.

Attribute	Notes
supplier_id (PK)	
supplier_name	
contact_name	Primary rep
phone, email	
address	
payment_terms	e.g., Net 30

5. PRODUCT

Every SKU sold or stocked at Jay-Mart.

Attribute	Notes
product_id (PK)	Surrogate key
sku	Unique, human-readable stock code
product_name	
description	
unit_price	Current selling price
cost_price	What Jay-Mart paid; used for margin reports
department_id (FK → DEPARTMENT)	Where the product is sold
supplier_id (FK → SUPPLIER)	Primary supplier
reorder_point	Quantity threshold triggering a new PO

<code>reorder_quantity</code>	How many units to order
<code>is_active</code>	Boolean; soft-deletes discontinued SKUs

Business rules: `unit_price` and `cost_price` must be > 0 . `sku` is globally unique. A product's department determines what sales category it rolls into.

6. INVENTORY

Current stock levels, separated from PRODUCT so a product could theoretically exist in multiple locations (e.g., stockroom vs. floor).

Attribute	Notes
<code>inventory_id</code> (PK)	
<code>product_id</code> (FK → PRODUCT)	
<code>location</code>	Enum: Floor, Stockroom
<code>quantity_on_hand</code>	Must be ≥ 0
<code>last_updated</code>	Timestamp of last stock change

Business rules: `quantity_on_hand` cannot go negative (enforced by constraint + application logic on sales). A reorder alert fires when `quantity_on_hand` $\leq$ `reorder_point` on the parent PRODUCT record.

7. PURCHASE_ORDER

Orders placed by Jay-Mart to restock from suppliers.

Attribute	Notes
<code>po_id</code> (PK)	
<code>supplier_id</code> (FK → SUPPLIER)	

<code>created_by</code> (FK → EMPLOYEE)	The manager who raised the PO
<code>order_date</code>	
<code>expected_delivery</code>	
<code>received_date</code>	Null until goods arrive
<code>status</code>	Enum: Draft, Sent, Partial, Received, Cancelled

8. PURCHASE_ORDER_LINE

Individual line items within a purchase order.

Attribute	Notes
<code>po_line_id</code> (PK)	
<code>po_id</code> (FK → PURCHASE_ORDER)	
<code>product_id</code> (FK → PRODUCT)	
<code>quantity_ordered</code>	
<code>quantity_received</code>	Incremented on partial deliveries
<code>unit_cost</code>	Locked-in cost at order time

Business rules: `quantity_received` cannot exceed `quantity_ordered`. On full receipt, the parent PO status advances to Received.

9. CUSTOMER

Registered loyalty-program members. Anonymous transactions do not require a record here.

Attribute	Notes
<code>customer_id</code> (PK)	

first_name,	
last_name	
email	Unique
phone	
date_of_birth	Optional; used for birthday promotions
loyalty_points	Running balance
registration_date	
is_active	Soft-delete flag

Business rules: `email` is the natural unique key used for login/lookup. `loyalty_points` cannot go below 0.

10. TRANSACTION

A sales event — one customer visit producing one receipt.

Attribute	Notes
<code>transaction_id</code> (PK)	
<code>transaction_date</code>	Datetime
<code>customer_id</code> (FK → CUSTOMER)	Nullable for anonymous sales
<code>cashier_id</code> (FK → EMPLOYEE)	The employee who processed it
<code>register_id</code>	Which till/lane
<code>subtotal</code>	Sum of line items before tax
<code>tax_amount</code>	
<code>total_amount</code>	subtotal + tax – discounts

<code>payment_method</code>	Enum: Cash, Credit, Debit, Gift Card, Loyalty Points
<code>status</code>	Enum: Completed, Voided, Partially Returned

Business rules: `total_amount = subtotal + tax_amount - discount_amount`. A voided transaction retains its record for audit purposes but is excluded from revenue reports.

11. TRANSACTION_LINE

Each individual product on a receipt.

Attribute	Notes
<code>line_id</code> (PK)	
<code>transaction_id</code> (FK → TRANSACTION)	
<code>product_id</code> (FK → PRODUCT)	
<code>quantity</code>	Must be ≥ 1
<code>unit_price</code>	Price at time of sale (may differ from current <code>PRODUCT.unit_price</code>)
<code>discount_applied</code>	Line-level discount amount
<code>line_total</code>	<code>(unit_price × quantity) - discount_applied</code>

Business rules: `unit_price` is snapshot-captured at sale time so historical transactions survive price changes. On insert, the corresponding INVENTORY record is decremented by `quantity`.

12. RETURN

Records merchandise brought back by customers.

Attribute	Notes
-----------	-------

<code>return_id</code> (PK)	
<code>original_transaction_id</code> (FK → TRANSACTION)	
<code>processed_by</code> (FK → EMPLOYEE)	
<code>return_date</code>	
<code>refund_method</code>	Enum: Original Payment, Store Credit, Loyalty Points
<code>total_refund</code>	

13. RETURN_LINE

Individual items within a return.

Attribute	Notes
<code>return_line_id</code> (PK)	
<code>return_id</code> (FK → RETURN)	
<code>transaction_line_id</code> (FK → TRANSACTION_LINE)	Links back to the original line
<code>quantity_returned</code>	Cannot exceed original <code>quantity</code>
<code>reason</code>	Enum: Defective, Wrong Item, Changed Mind, Other

Business rules: `quantity_returned` ≤ `quantity` on the original TRANSACTION_LINE. On insert, the corresponding INVENTORY record is incremented.

Key Cross-Entity Constraints Summary

Constraint	Enforcement
Inventory never negative	CHECK (<code>quantity_on_hand</code> >= 0)
Sale price locked at transaction time	Snapshot <code>unit_price</code> into TRANSACTION_LINE

Employee can't self-manage	<code>CHECK (employee_id <> manager_id)</code>
Return qty $\leq$ original sale qty	Application logic + DB constraint
Active schedules exclude terminated staff	Filtered via <code>termination_date IS NULL</code>
Loyalty points never negative	<code>CHECK (loyalty_points >= 0)</code>
Non-negative prices	<code>CHECK (unit_price > 0 AND cost_price > 0)</code>

This structure gives Jay-Mart full coverage across HR, inventory, procurement, point-of-sale, and customer loyalty — with clean foreign key chains that support reporting across all domains. The next step would be translating this into a formal **Entity-Relationship Diagram (ERD)** and then DDL (`CREATE TABLE` statements).

My analysis of the first input/output:

I think some entities were unnecessary. For the next prompt, I plan to edit the original prompt and specify the important entities. Some constraints were missed.

Second input prompt: “Jay-Mart is a small (fictional) multi-department retail store with approximately 50 employees, several product departments (e.g., Electronics, Clothing, Grocery), and a growing customer base. I need you to design a database to support day-to-day operations, including employee management, inventory tracking, customer records, and sales transactions. First, describe the corporate and operational structure of a small department store by identifying entity types the business manages, data stored per entity, specifying primary and foreign key candidates, and relevant business rules or constraints.

Make sure to explicitly identify primary keys and foreign keys
use simple entity examples like, department, category, employee, attendance, product, inventory, customer, transaction, transaction_line, payment, promotion, product_promotion. Provide important constraints like NOT NULL, UNIQUE”

Output:

Here's a full corporate and operational structure breakdown for Jay-Mart, followed by an entity-relationship diagram.

Jay-Mart: Entity & Operational Structure

Entity inventory

department — the top-level organizational unit (Electronics, Clothing, Grocery). Each department has a manager drawn from the employee pool, giving us a self-referencing FK through **employee**.

category — a product sub-grouping *within* a department (e.g., "Phones" under Electronics, "Dairy" under Grocery). A category belongs to exactly one department.

employee — any person on payroll. Tied to a department, carries role/salary info, and has a hire date. One employee can manage one department (creating a circular reference resolved by making **manager_id** nullable).

attendance — a daily punch record per employee. Tracks clock-in/out and a status flag (present, absent, late). An employee can have many attendance records; each record belongs to exactly one employee.

product — a sellable SKU. Belongs to one category. Carries its base price and unit of measure. The combination of (**name**, **category_id**) is unique — the same product name can exist in different categories but not twice in the same one.

inventory — a one-to-one extension of product that tracks quantity on hand, reorder threshold, and last-restocked date. Separating this from **product** keeps the product table lean and makes inventory auditing easier.

customer — a registered shopper. Email must be unique. Loyalty tier (standard, silver, gold) drives discount eligibility.

transaction — a completed sale session. Belongs to one customer; processed by one employee. Records the date/time and overall total.

transaction_line — one row per product per transaction (the receipt line). Stores quantity and the unit price *at time of sale* (important: prices change, but the historical record must not).

payment — records how a transaction was settled. A transaction can be split across payment methods (cash + card), so this is a child of **transaction**, not a 1-to-1.

promotion — a store-wide or department-wide discount campaign with a validity window and a discount percentage.

product_promotion — a junction table linking specific products to active promotions (many-to-many).

Business rules & constraints

- An employee must belong to a department (**NOT NULL**), but a department's manager field is nullable until assigned.
- A product must belong to a category; a category must belong to a department — the chain is fully non-nullable.
- `transaction_line.unit_price` is stored explicitly (not looked up from `product`) to preserve historical accuracy.
- `inventory.quantity_on_hand` must be ≥ 0 (a **CHECK** constraint).
- `payment.amount` must be > 0 .
- `promotion.discount_pct` must be between 0 and 1 exclusive.
- A product cannot be linked to the same promotion twice (**UNIQUE** on `product_promotion(product_id, promotion_id)`).

Now here's the full ERD:--- ***unpastable ER diagram***

Key relationships summary

Relationship	Cardinality	FK lives in
Department → Category	1 : many	<code>category.department_id</code>
Department → Employee	1 : many	<code>employee.department_id</code>
Department → Manager	1 : 0-or-1	<code>department.manager_id</code>
Employee → Attendance	1 : many	<code>attendance.employee_id</code>
Category → Product	1 : many	<code>product.category_id</code>
Product → Inventory	1 : 1	<code>inventory.product_id</code>
Customer → Transaction	1 : many	<code>transaction.customer_id</code>
Employee → Transaction	1 : many	<code>transaction.employee_id</code>
Transaction → Transaction_line	1 : many	<code>transaction_line.transaction_id</code>
Transaction → Payment	1 : many	<code>payment.transaction_id</code>

Product ↔ Promotion many : `product_promotion(product_id,`
many `promotion_id)`

Notable design decisions

Historical pricing — `transaction_line.unit_price` is denormalized by design. When a product's price changes, past receipts remain accurate.

Inventory separation — keeping `inventory` as a child of `product` (rather than columns on `product`) makes it easy to add warehouse locations or multi-store support later without touching the core product table.

Nullable manager — `department.manager_id` is `NULL`-able to break the circular dependency at insert time (you need the employee row before you can point the department at them).

Promotion junction — `product_promotion` has a composite `UNIQUE(product_id, promotion_id)` constraint so the same deal can't be double-applied to one SKU.

Analysis of Second Output:

-gave me an ER diagram when I didn't ask for one

-I forgot “supplier”

-I'm not sure if I should prompt another answer without the ER diagram.

-didn't specify entity attributes at all outside of ER diagram

Improvements: relationships are clearer, entities are simpler.

Third prompt:

Goal is to provide all entities and relationships without the ER diagram

“add this entity “supplier” also can you give me all entities and their relationships BUT withhold from providing an ER diagram at this moment”

Output:

Supplier Entity

supplier — represents a vendor or company that supplies products to Jay-Mart. A product can come from one supplier, and a supplier can provide many products. Optionally, a supplier can be associated with specific categories they specialize in.

Attributes:

- `supplier_id` — INT, **PK**, auto-increment
- `name` — VARCHAR, NOT NULL, UNIQUE

- **contact_name** — VARCHAR, NOT NULL (primary point of contact)
- **email** — VARCHAR, NOT NULL, UNIQUE
- **phone** — VARCHAR, NOT NULL
- **address** — VARCHAR
- **city** — VARCHAR
- **country** — VARCHAR, NOT NULL
- **status** — VARCHAR, NOT NULL, DEFAULT 'active' (active / inactive)

FK added to **product**: `product.supplier_id` → `supplier.supplier_id`

All 13 Entities & Their Relationships

1. Department

Stores the store's operational divisions (Electronics, Clothing, Grocery).

- **PK:** `department_id`
- **FK:** `manager_id` → `employee.employee_id` (nullable — one employee optionally manages it)
- **Relationships:**
 - Has many `category` records
 - Has many `employee` records
 - Is managed by one `employee` (optional)

2. Category

A product sub-grouping within a department.

- **PK:** `category_id`
- **FK:** `department_id` → `department.department_id` NOT NULL
- **Constraints:** `UNIQUE(name, department_id)` — same category name can't appear twice in the same department
- **Relationships:**
 - Belongs to one `department`
 - Has many `product` records

3. Employee

Any person on Jay-Mart's payroll.

- **PK:** `employee_id`
 - **FK:** `department_id` → `department.department_id` NOT NULL
 - **Constraints:** `email` UNIQUE, NOT NULL; `salary` CHECK > 0
 - **Relationships:**
 - Belongs to one `department`
 - Has many `attendance` records
 - Can process many `transaction` records
 - Can optionally manage one `department`
-

4. Attendance

Daily clock-in/clock-out record per employee.

- **PK:** `attendance_id`
 - **FK:** `employee_id` → `employee.employee_id` NOT NULL
 - **Constraints:** `UNIQUE(employee_id, work_date)` — one record per employee per day; `status` CHECK IN ('present', 'absent', 'late')
 - **Relationships:**
 - Belongs to one `employee`
-

5. Supplier

A vendor that supplies products to Jay-Mart.

- **PK:** `supplier_id`
 - **Constraints:** `name` UNIQUE NOT NULL; `email` UNIQUE NOT NULL; `status` CHECK IN ('active', 'inactive')
 - **Relationships:**
 - Supplies many `product` records
-

6. Product

A sellable SKU available in the store.

- **PK:** `product_id`
- **FK:** `category_id` → `category.category_id` NOT NULL
- **FK:** `supplier_id` → `supplier.supplier_id` NOT NULL

- **Constraints:** `UNIQUE(name, category_id); price CHECK > 0`
 - **Relationships:**
 - Belongs to one `category`
 - Supplied by one `supplier`
 - Has one `inventory` record
 - Appears in many `transaction_line` records
 - Linked to many `promotion` records via `product_promotion`
-

7. Inventory

Tracks stock levels for a product.

- **PK:** `inventory_id`
 - **FK:** `product_id` → `product.product_id` NOT NULL, UNIQUE (enforces 1-to-1)
 - **Constraints:** `quantity_on_hand CHECK ≥ 0; reorder_threshold CHECK ≥ 0`
 - **Relationships:**
 - Belongs to exactly one `product`
-

8. Customer

A registered shopper.

- **PK:** `customer_id`
 - **Constraints:** `email` UNIQUE NOT NULL; `loyalty_tier` CHECK IN ('standard', 'silver', 'gold')
 - **Relationships:**
 - Places many `transaction` records
-

9. Transaction

A completed sale session at the register.

- **PK:** `transaction_id`
- **FK:** `customer_id` → `customer.customer_id` NOT NULL
- **FK:** `employee_id` → `employee.employee_id` NOT NULL
- **Constraints:** `total_amount` CHECK ≥ 0
- **Relationships:**

- Belongs to one **customer**
 - Processed by one **employee**
 - Has many **transaction_line** records
 - Has many **payment** records
-

10. **Transaction_line**

One line item on a receipt — one product per row.

- **PK:** **line_id**
 - **FK:** **transaction_id** → **transaction.transaction_id** NOT NULL
 - **FK:** **product_id** → **product.product_id** NOT NULL
 - **Constraints:** **quantity** CHECK > 0; **unit_price** CHECK > 0;
UNIQUE(transaction_id, product_id) — same product can't appear twice on one receipt
 - **Relationships:**
 - Belongs to one **transaction**
 - References one **product**
-

11. **payment**

Records how a transaction was settled (supports split payments).

- **PK:** **payment_id**
 - **FK:** **transaction_id** → **transaction.transaction_id** NOT NULL
 - **Constraints:** **method** CHECK IN ('cash', 'card', 'mobile', 'voucher'); **amount** CHECK > 0
 - **Relationships:**
 - Belongs to one **transaction**
-

12. **Promotion**

A discount campaign with a validity window.

- **PK:** **promotion_id**
- **Constraints:** **name** UNIQUE NOT NULL; **discount_pct** CHECK > 0 AND < 1;
end_date CHECK > **start_date**

- **Relationships:**
 - Linked to many `product` records via `product_promotion`

13. `Product_promotion`

Junction table resolving the many-to-many between products and promotions.

- **PK:** composite (`product_id`, `promotion_id`)
- **FK:** `product_id` → `product.product_id` NOT NULL
- **FK:** `promotion_id` → `promotion.promotion_id` NOT NULL
- **Constraints:** `UNIQUE(product_id, promotion_id)` — a product can't be linked to the same promotion twice
- **Relationships:**
 - Bridges `product` and `promotion`

Relationship map at a glance

From	To	Type	FK location
<code>department</code>	<code>category</code>	1 : many	<code>category.department_id</code>
<code>department</code>	<code>employee</code>	1 : many	<code>employee.department_id</code>
<code>employee</code>	<code>department</code>	0-or-1 : 1	<code>department.manager_id</code>
<code>employee</code>	<code>attendance</code>	1 : many	<code>attendance.employee_id</code>
<code>employee</code>	<code>transaction</code>	1 : many	<code>transaction.employee_id</code>
<code>supplier</code>	<code>product</code>	1 : many	<code>product.supplier_id</code>
<code>category</code>	<code>product</code>	1 : many	<code>product.category_id</code>
<code>product</code>	<code>inventory</code>	1 : 1	<code>inventory.product_id</code>
<code>product</code>	<code>transaction_line</code>	1 : many	<code>transaction_line.product_id</code>

product	product_promotion	1 : many	product_promotion.product_id
customer	transaction	1 : many	transaction.customer_id
transaction	transaction_line	1 : many	transaction_line.transaction_id
transaction	payment	1 : many	payment.transaction_id
promotion	product_promotion	1 : many	product_promotion.promotion_id

Output Analysis:

I noticed some attributes seemed extra and unnecessary so I want to remove some without affecting functionality. Also not all attributes were provided.

Fourth Prompt:

“for supplier can we remove address, city, country, and status without affecting anything?”

Output:

Yes, absolutely — removing those four columns has no downstream impact whatsoever. Here's why:

Why it's safe to remove them

- **address, city, country** — these are purely informational/contact fields. No other entity has a FK pointing to them, no business rule in the schema depends on a supplier's location, and no junction table references them. They're descriptive metadata, not structural.
- **status** — none of the other 12 entities reference a supplier's status in any constraint or relationship. There's no rule like "a product can only exist if its supplier is active." If you wanted that kind of enforcement later, you'd add it back then.

Nothing that breaks

Concern	Safe?	Reason
Foreign keys in other tables	Yes	No other table references these columns

<code>product.supplier_id</code> FK	Yes	That FK points to <code>supplier_id</code> (the PK), not any of the removed columns
Business rules / constraints	Yes	No CHECK or UNIQUE constraint elsewhere depends on them
Junction tables	Yes	<code>product_promotion</code> doesn't involve supplier at all

Revised `supplier` entity

- `supplier_id` — INT, **PK**, auto-increment
- `name` — VARCHAR, NOT NULL, UNIQUE
- `contact_name` — VARCHAR, NOT NULL
- `email` — VARCHAR, NOT NULL, UNIQUE
- `phone` — VARCHAR, NOT NULL

Clean and minimal — just enough to identify the supplier and reach them. You can always add those columns back later if Jay-Mart expands and needs vendor logistics tracking.

Fifth prompt:

“briefly can you do all these steps, write down:

1. name all entities
2. specify all attributes per entity
3. pk and fk candidates
4. business rules or constraints”

Output:

Jay-Mart Database — Full Specification

1. Entities

`department, category, employee, attendance, supplier, product, inventory, customer, transaction, transaction_line, payment, promotion, product_promotion`

2–4. Attributes, Keys & Constraints

department

- `department_id` INT — **PK**
- `name` VARCHAR — NOT NULL, UNIQUE
- `manager_id` INT — **FK** → **employee.employee_id**, nullable

category

- `category_id` INT — **PK**
- `name` VARCHAR — NOT NULL
- `department_id` INT — **FK** → **department.department_id**, NOT NULL
- UNIQUE(`name`, `department_id`)

employee

- `employee_id` INT — **PK**
- `first_name` VARCHAR — NOT NULL
- `last_name` VARCHAR — NOT NULL
- `email` VARCHAR — NOT NULL, UNIQUE
- `role` VARCHAR — NOT NULL
- `salary` DECIMAL — NOT NULL, CHECK > 0
- `hire_date` DATE — NOT NULL
- `department_id` INT — **FK** → **department.department_id**, NOT NULL

attendance

- `attendance_id` INT — **PK**
- `employee_id` INT — **FK** → **employee.employee_id**, NOT NULL
- `work_date` DATE — NOT NULL
- `clock_in` TIME — NOT NULL
- `clock_out` TIME — nullable
- `status` VARCHAR — NOT NULL, CHECK IN ('present', 'absent', 'late')
- UNIQUE(`employee_id`, `work_date`)

supplier

- `supplier_id` INT — **PK**
- `supplier_name` VARCHAR — NOT NULL, UNIQUE

- `contact_name` VARCHAR — NOT NULL
- `email` VARCHAR — NOT NULL, UNIQUE
- `phone` VARCHAR — NOT NULL

product

- `product_id` INT — **PK**
- `name` VARCHAR — NOT NULL
- `price` DECIMAL — NOT NULL, CHECK > 0
- `unit_of_measure` VARCHAR — NOT NULL
- `category_id` INT — **FK** → **category.category_id**, NOT NULL
- `supplier_id` INT — **FK** → **supplier.supplier_id**, NOT NULL
- UNIQUE(`name`, `category_id`)

inventory

- `inventory_id` INT — **PK**
- `product_id` INT — **FK** → **product.product_id**, NOT NULL, UNIQUE
- `quantity_on_hand` INT — NOT NULL, CHECK ≥ 0
- `reorder_threshold` INT — NOT NULL, CHECK ≥ 0
- `last_restocked` DATE — nullable

customer

- `customer_id` INT — **PK**
- `first_name` VARCHAR — NOT NULL
- `last_name` VARCHAR — NOT NULL
- `email` VARCHAR — NOT NULL, UNIQUE
- `phone` VARCHAR — nullable
- `loyalty_tier` VARCHAR — NOT NULL, DEFAULT 'standard', CHECK IN ('standard', 'silver', 'gold')
- `registered_at` DATE — NOT NULL

transaction

- `transaction_id` INT — **PK**
- `customer_id` INT — **FK** → **customer.customer_id**, NOT NULL
- `employee_id` INT — **FK** → **employee.employee_id**, NOT NULL

- `transaction_dt` TIMESTAMP — NOT NULL
- `total_amount` DECIMAL — NOT NULL, CHECK ≥ 0

`transaction_line`

- `line_id` INT — **PK**
- `transaction_id` INT — **FK** → `transaction.transaction_id`, NOT NULL
- `product_id` INT — **FK** → `product.product_id`, NOT NULL
- `quantity` INT — NOT NULL, CHECK > 0
- `unit_price` DECIMAL — NOT NULL, CHECK > 0
- UNIQUE(`transaction_id`, `product_id`)

`payment`

- `payment_id` INT — **PK**
- `transaction_id` INT — **FK** → `transaction.transaction_id`, NOT NULL
- `method` VARCHAR — NOT NULL, CHECK IN ('cash', 'card', 'mobile', 'voucher')
- `amount` DECIMAL — NOT NULL, CHECK > 0

`promotion`

- `promotion_id` INT — **PK**
- `name` VARCHAR — NOT NULL, UNIQUE
- `discount_pct` DECIMAL — NOT NULL, CHECK > 0 AND < 1
- `start_date` DATE — NOT NULL
- `end_date` DATE — NOT NULL, CHECK $> \text{start_date}$

`product_promotion`

- `product_id` INT — **PK/FK** → `product.product_id`, NOT NULL
- `promotion_id` INT — **PK/FK** → `promotion.promotion_id`, NOT NULL
- Composite PK(`product_id`, `promotion_id`)

ANALYSIS:

We are going to move on with these outputs to part 2 as the ai agent gave me exactly what i asked for.

Finalized requirements: The database will manage employees, departments, suppliers, products, inventory, customers, sales transactions, payments, promotions, and attendance records. The system will allow Jay-Mart to track inventory levels, customer purchases,

employee activity, and promotional campaigns while maintaining data integrity through relational constraints.

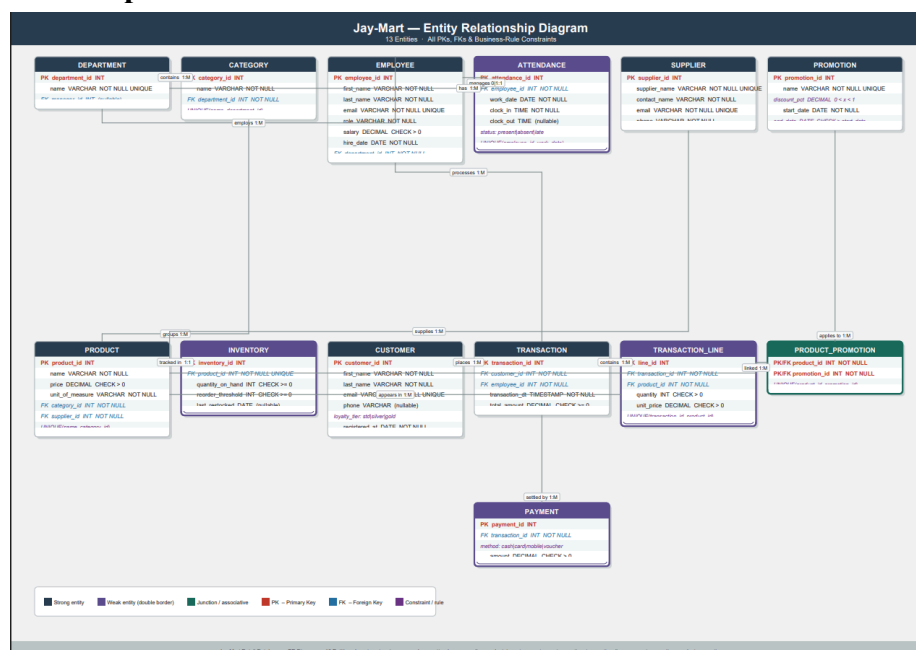
Business Rules and Integrity Constraints:

- Employee email addresses must be unique and cannot be null.**
- Customer email addresses must be unique and cannot be null.**
- Product prices and payment amounts must be greater than zero.**
- Inventory quantity_on_hand and reorder_threshold values cannot be negative.**
- A product must belong to exactly one category and one supplier.**
- Attendance records must allow only the following status values: present, absent, or late.**
- Promotion discount percentages must be greater than 0 and less than 1.**
- The same product cannot appear twice within the same transaction.**
- A product cannot be linked to the same promotion more than once.**

PART II

Initial prompt: “using these Business Rules and Integrity Constraints: -Employee email addresses must be unique and cannot be null. -Customer email addresses must be unique and cannot be null. -Product prices and payment amounts must be greater than zero. -Inventory quantity_on_hand and reorder_threshold values cannot be negative. -A product must belong to exactly one category and one supplier. -Attendance records must allow only the following status values: present, absent, or late. -Promotion discount percentages must be greater than 0 and less than 1. -The same product cannot appear twice within the same transaction. -A product cannot be linked to the same promotion more than once. generate a downloadable ER diagram make sure to provide all entities, all relationships with the correct cardinality, pks and fks are correctly identified, and any weak entities are modeled appropriately”

First output:



ANALYSIS: This showed me I can add more rules and integrity constraints and also i think i made an error when i asked for a downloadable one, the diagram is practically unreadable and just overall messy, but did follow my initial requirements!

Second prompt: “using these Business Rules and Integrity Constraints AND relationships:
-Employee email addresses must be unique and cannot be null. -Customer email addresses must be unique and cannot be null. -Product prices and payment amounts must be greater than zero. -Inventory quantity_on_hand and reorder_threshold values cannot be negative. -A product must belong to exactly one category and one supplier. -Attendance records must allow only the following status values: present, absent, or late. -Promotion discount percentages must be greater than 0 and less than 1. relationships: -each employee belongs to one department -sales are handled by one employee -a department can contain many categories and employees. -a category belongs to one department and can contain many

products. -a supplier can supply many products. -product belongs to one category and one supplier. -each product has one inventory record. -customers can make many transactions. -employees can process many transactions. -A transaction can contain many transaction line items and multiple payments. can you build a conceptual data model, specifically an ER diagram, easy to read and comprehend. *make sure to provide all entities, all relationships with the correct cardinality, pks and fks are correctly identified, and any weak entities are modeled appropriately**

Output:

Jay-Mart conceptual entity-relationship diagram showing 13 entities and their relationships with cardinality notation



A few things worth noting about the diagram:

Weak entities — **attendance**, **transaction_line**, and **payment** are modeled as weak entities. They have no meaningful existence without their parent (**employee**,

transaction respectively) and their PKs depend on the parent's FK to be fully identified.

product_promotion is a pure junction/associative entity resolving the many-to-many between **product** and **promotion**. Its identity is entirely composite — both columns are simultaneously FK and PK.

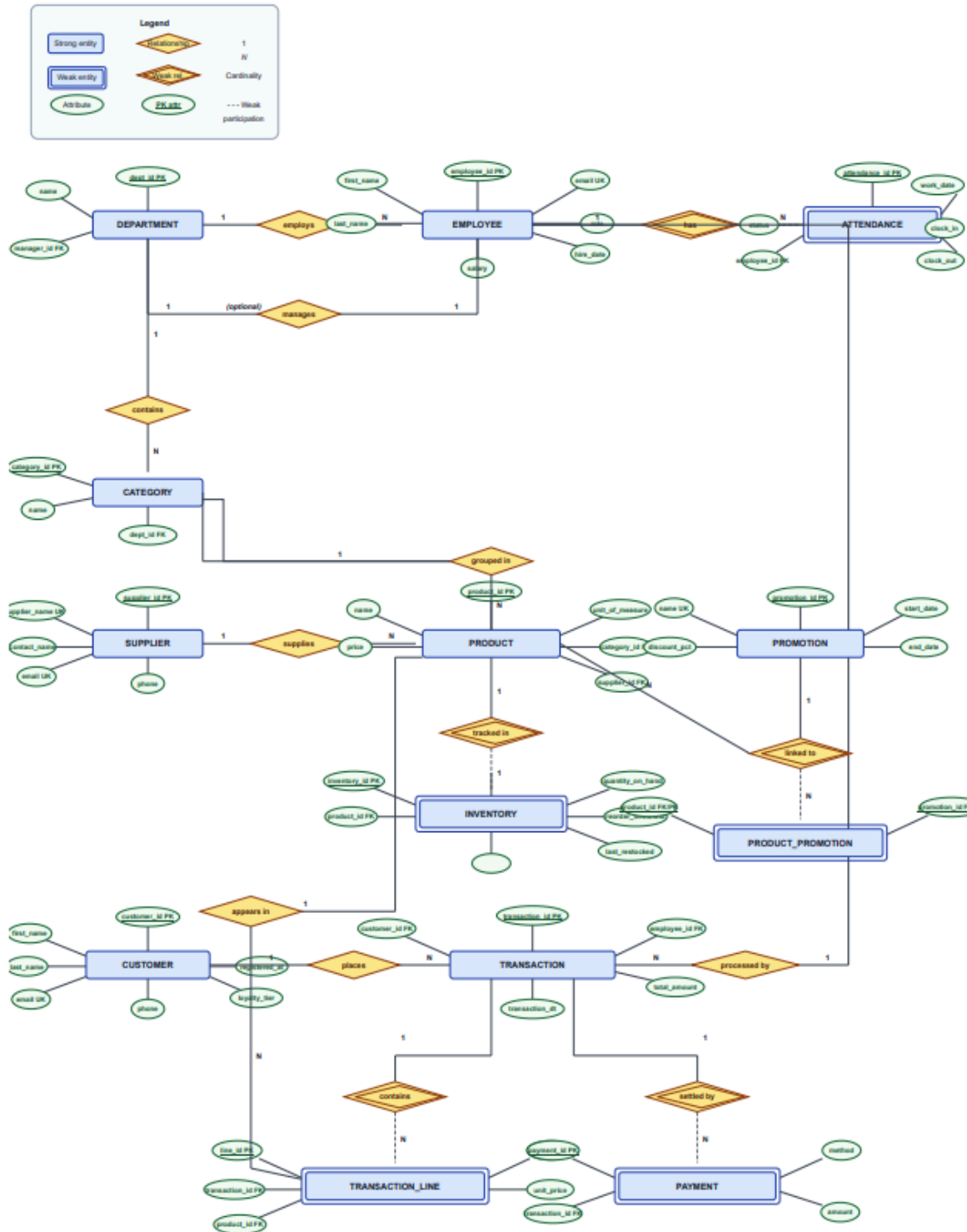
The **department** ↔ **employee** double relationship — there are two distinct lines between them: one for "employs" (1 department to many employees) and one for "managed by" (a department optionally has one manager who is an employee). This reflects the nullable **manager_id** FK on **department**.

Cardinality notation used — crow's foot notation throughout: **||** = exactly one, **o|** = zero or one, **o{** = zero or many, **|{** = one or many.

ANALYSIS: while it did follow my steps i think i can further request a more readable diagram and a different notation.

Third Prompt: “can you use chen's notation instead to better readability, and clearly model weak entities on the diagram. all labels should be clear. also can you make the letters darker, as its very hard to see”

Output is down below:



Here's a breakdown of the Chen notation conventions used in the diagram:

Shapes — rectangles are strong entities, double-bordered rectangles are weak entities, diamonds are relationships, double-bordered diamonds are identifying relationships (for weak entities), and ellipses are attributes. Underlined attributes denote primary keys.

Weak entities — **attendance**, **inventory**, **transaction_line**, **payment**, and **product_promotion** are all modeled as weak entities with double borders. Their identifying relationships use double-bordered diamonds, and their participation lines are dashed to distinguish them from strong entity connections.

Cardinality — **1** and **N** labels sit on each side of every relationship line, directly indicating whether the participation is one or many. The **manages** relationship between **department** and **employee** is marked optional since **manager_id** is nullable.

product_promotion — modeled as a weak associative entity whose entire identity is the composite of **product_id** and **promotion_id**, both marked as FK/PK to reflect that.

ANALYSIS: this diagram is so much better.

AI Tool Consistency Reflection:

Using the same Gen AI in both parts is important because it not only preserves the conversational context but ensures consistency throughout the entire database building process. The agent already has access to the finalized responsibilities and business rules developed in Part 1, which allows it to generate an ER diagram that accurately reflects those decisions without re-interpreting. It also helps maintain consistent terminology, such as entity and attribute name. Switching tools mid project introduces risks such as losing prior context, reintroducing unnecessary complexity, or generating conflicting assumptions about relationships like cardinality or key structure. This also can result in an ER diagram that does not match the finalized requirements, forcing additional rework.

PART III

3a.

First prompt: “using my finalized requirements: “ -Employee email addresses must be unique and cannot be null. -Customer email addresses must be unique and cannot be null. -Product prices and payment amounts must be greater than zero. -Inventory quantity_on_hand and reorder_threshold values cannot be negative. -A product must belong to exactly one category and one supplier. -Attendance records must allow only the following status values: present, absent, or late. -Promotion discount percentages must be greater than 0 and less than 1. relationships: -each employee belongs to one department -sales are handled by one employee -a department can contain many categories and employees. -a category belongs to one department and can contain many products. -a supplier can supply many products. -product belongs to one category and one supplier. -each product has one inventory record. -customers can make many transactions. -employees can process many transactions. -A transaction can contain many transaction line items and multiple payments. can you build a conceptual data model, specifically an ER diagram, easy to read and comprehend. *make sure to provide all entities, all relationships with the correct cardinality, pks and fks are correctly identified, and any weak entities are modeled appropriately*” AND the recent ER diagram above...

generate complete MySQL DDL code to create all tables, primary keys, foreign keys, UNIQUE constraints, NOT NULL constraints, CHECK constraints, and relationship definitions. Use proper table creation order to avoid foreign key errors and include AUTO_INCREMENT for all primary key IDs where appropriate.”

OUTPUT:

```
-- =====
-- JAY-MART – Complete MySQL DDL
-- Tables are created in dependency order so every FK
-- references a table that already exists.
-- =====

-- =====
-- 1. SUPPLIER
-- No dependencies – created first.
-- =====

CREATE TABLE supplier (
  supplier_id INT NOT NULL AUTO_INCREMENT,
  supplier_name VARCHAR(120) NOT NULL,
  contact_name VARCHAR(120) NOT NULL,
  email VARCHAR(180) NOT NULL,
  phone VARCHAR(30) NOT NULL,

  CONSTRAINT pk_supplier PRIMARY KEY (supplier_id),
  CONSTRAINT uq_supplier_name UNIQUE (supplier_name),
  CONSTRAINT uq_supplier_email UNIQUE (email)
);
```

```
-- =====  
-- 2. EMPLOYEE (forward-declared before DEPARTMENT  
-- because DEPARTMENT.manager_id references EMPLOYEE)  
-- =====
```

```
CREATE TABLE employee (  
    employee_id INT NOT NULL AUTO_INCREMENT,  
    first_name VARCHAR(80) NOT NULL,  
    last_name VARCHAR(80) NOT NULL,  
    email VARCHAR(180) NOT NULL,  
    role VARCHAR(60) NOT NULL,  
    salary DECIMAL(10,2) NOT NULL,  
    hire_date DATE NOT NULL,  
    department_id INT NOT NULL, -- FK added via ALTER below  
  
    CONSTRAINT pk_employee PRIMARY KEY (employee_id),  
    CONSTRAINT uq_employee_email UNIQUE (email),  
    CONSTRAINT ck_employee_salary CHECK (salary > 0)  
);
```

```
-- =====  
-- 3. DEPARTMENT  
-- manager_id references EMPLOYEE (nullable – no manager yet).  
-- =====
```

```
CREATE TABLE department (  
    department_id INT NOT NULL AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    manager_id INT NULL, -- optional manager  
  
    CONSTRAINT pk_department PRIMARY KEY (department_id),  
    CONSTRAINT uq_department_name UNIQUE (name),  
    CONSTRAINT fk_dept_manager FOREIGN KEY (manager_id)  
        REFERENCES employee (employee_id)  
        ON UPDATE CASCADE  
        ON DELETE SET NULL  
);
```

```
-- =====  
-- 4. Back-fill the FK from EMPLOYEE → DEPARTMENT  
-- (needed because the two tables reference each other)  
-- =====
```

```
ALTER TABLE employee  
    ADD CONSTRAINT fk_emp_department  
        FOREIGN KEY (department_id)  
        REFERENCES department (department_id)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT;
```

```
-- =====  
-- 5. ATTENDANCE (weak entity – depends on EMPLOYEE)  
-- =====
```

```
CREATE TABLE attendance (  
    attendance_id INT NOT NULL AUTO_INCREMENT,  
    employee_id INT NOT NULL,  
    work_date DATE NOT NULL,
```

```

clock_in    TIME    NOT NULL,
clock_out   TIME     NULL,      -- nullable: employee may not have clocked out yet
status     VARCHAR(10) NOT NULL,

CONSTRAINT pk_attendance    PRIMARY KEY (attendance_id),
CONSTRAINT uq_attendance_daily UNIQUE    (employee_id, work_date),
CONSTRAINT fk_attend_employee FOREIGN KEY (employee_id)
    REFERENCES employee (employee_id)
    ON UPDATE CASCADE
    ON DELETE CASCADE,
CONSTRAINT ck_attend_status  CHECK (status IN ('present', 'absent', 'late')),
CONSTRAINT ck_attend_clockout CHECK (clock_out IS NULL OR clock_out >= clock_in)
);

-- =====
-- 6. CATEGORY (depends on DEPARTMENT)
-- =====

CREATE TABLE category (
    category_id INT      NOT NULL AUTO_INCREMENT,
    name        VARCHAR(100) NOT NULL,
    department_id INT     NOT NULL,

    CONSTRAINT pk_category    PRIMARY KEY (category_id),
    CONSTRAINT uq_category_dept UNIQUE    (name, department_id),
    CONSTRAINT fk_cat_department FOREIGN KEY (department_id)
        REFERENCES department (department_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
);

-- =====
-- 7. PRODUCT (depends on CATEGORY and SUPPLIER)
-- =====

CREATE TABLE product (
    product_id INT      NOT NULL AUTO_INCREMENT,
    name        VARCHAR(150) NOT NULL,
    price       DECIMAL(10,2) NOT NULL,
    unit_of_measure VARCHAR(40) NOT NULL,
    category_id INT     NOT NULL,
    supplier_id INT     NOT NULL,

    CONSTRAINT pk_product     PRIMARY KEY (product_id),
    CONSTRAINT uq_product_cat  UNIQUE    (name, category_id),
    CONSTRAINT fk_prod_category FOREIGN KEY (category_id)
        REFERENCES category (category_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT fk_prod_supplier FOREIGN KEY (supplier_id)
        REFERENCES supplier (supplier_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT ck_product_price CHECK (price > 0)
);

```

```
-- =====  
-- 8. INVENTORY (weak entity – 1-to-1 with PRODUCT)  
-- =====
```

```
CREATE TABLE inventory (  
    inventory_id INT NOT NULL AUTO_INCREMENT,  
    product_id INT NOT NULL,  
    quantity_on_hand INT NOT NULL DEFAULT 0,  
    reorder_threshold INT NOT NULL DEFAULT 0,  
    last_restocked DATE NULL,  
  
    CONSTRAINT pk_inventory PRIMARY KEY (inventory_id),  
    CONSTRAINT uq_inventory_product UNIQUE (product_id), -- enforces 1-to-1  
    CONSTRAINT fk_inv_product FOREIGN KEY (product_id)  
        REFERENCES product (product_id)  
        ON UPDATE CASCADE  
        ON DELETE CASCADE,  
    CONSTRAINT ck_inv_qty CHECK (quantity_on_hand >= 0),  
    CONSTRAINT ck_inv_reorder CHECK (reorder_threshold >= 0)  
);
```

```
-- =====  
-- 9. CUSTOMER  
-- No FK dependencies on other domain tables.  
-- =====
```

```
CREATE TABLE customer (  
    customer_id INT NOT NULL AUTO_INCREMENT,  
    first_name VARCHAR(80) NOT NULL,  
    last_name VARCHAR(80) NOT NULL,  
    email VARCHAR(180) NOT NULL,  
    phone VARCHAR(30) NULL,  
    loyalty_tier VARCHAR(10) NOT NULL DEFAULT 'standard',  
    registered_at DATE NOT NULL,  
  
    CONSTRAINT pk_customer PRIMARY KEY (customer_id),  
    CONSTRAINT uq_customer_email UNIQUE (email),  
    CONSTRAINT ck_customer_loyalty CHECK (loyalty_tier IN ('standard', 'silver', 'gold'))  
);
```

```
-- =====  
-- 10. TRANSACTION (depends on CUSTOMER and EMPLOYEE)  
-- =====
```

```
CREATE TABLE transaction (  
    transaction_id INT NOT NULL AUTO_INCREMENT,  
    customer_id INT NOT NULL,  
    employee_id INT NOT NULL,  
    transaction_dt DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    total_amount DECIMAL(12,2) NOT NULL DEFAULT 0.00,  
  
    CONSTRAINT pk_transaction PRIMARY KEY (transaction_id),  
    CONSTRAINT fk_txn_customer FOREIGN KEY (customer_id)  
        REFERENCES customer (customer_id)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT,  
    CONSTRAINT fk_txn_employee FOREIGN KEY (employee_id)
```

```

REFERENCES employee (employee_id)
ON UPDATE CASCADE
ON DELETE RESTRICT,
CONSTRAINT ck_txn_total    CHECK (total_amount >= 0)
);

-- =====
-- 11. TRANSACTION_LINE (weak entity – depends on TRANSACTION and PRODUCT)
-- =====
CREATE TABLE transaction_line (
  line_id    INT      NOT NULL AUTO_INCREMENT,
  transaction_id INT    NOT NULL,
  product_id INT      NOT NULL,
  quantity   INT      NOT NULL,
  unit_price DECIMAL(10,2) NOT NULL,

  CONSTRAINT pk_txn_line    PRIMARY KEY (line_id),
  CONSTRAINT uq_txn_line_product UNIQUE (transaction_id, product_id),
  CONSTRAINT fk_line_transaction FOREIGN KEY (transaction_id)
    REFERENCES transaction (transaction_id)
    ON UPDATE CASCADE
    ON DELETE CASCADE,
  CONSTRAINT fk_line_product FOREIGN KEY (product_id)
    REFERENCES product (product_id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
  CONSTRAINT ck_line_quantity CHECK (quantity > 0),
  CONSTRAINT ck_line_unit_price CHECK (unit_price > 0)
);

-- =====
-- 12. PAYMENT (weak entity – depends on TRANSACTION)
-- =====
CREATE TABLE payment (
  payment_id INT      NOT NULL AUTO_INCREMENT,
  transaction_id INT    NOT NULL,
  method     VARCHAR(20) NOT NULL,
  amount     DECIMAL(10,2) NOT NULL,

  CONSTRAINT pk_payment    PRIMARY KEY (payment_id),
  CONSTRAINT fk_pay_transaction FOREIGN KEY (transaction_id)
    REFERENCES transaction (transaction_id)
    ON UPDATE CASCADE
    ON DELETE CASCADE,
  CONSTRAINT ck_pay_method CHECK (method IN ('cash', 'card', 'mobile', 'voucher')),
  CONSTRAINT ck_pay_amount CHECK (amount > 0)
);

-- =====
-- 13. PROMOTION
-- No FK dependencies on other domain tables.
-- =====
CREATE TABLE promotion (
  promotion_id INT      NOT NULL AUTO_INCREMENT,

```

```

name      VARCHAR(150) NOT NULL,
discount_pct DECIMAL(5,4) NOT NULL,
start_date DATE      NOT NULL,
end_date  DATE      NOT NULL,

CONSTRAINT pk_promotion      PRIMARY KEY (promotion_id),
CONSTRAINT uq_promotion_name UNIQUE   (name),
CONSTRAINT ck_promo_discount CHECK (discount_pct > 0 AND discount_pct < 1),
CONSTRAINT ck_promo_dates    CHECK (end_date > start_date)
);

-- =====
-- 14. PRODUCT_PROMOTION (weak/junction entity – many-to-many)
-- =====

CREATE TABLE product_promotion (
  product_id INT NOT NULL,
  promotion_id INT NOT NULL,

  CONSTRAINT pk_product_promotion PRIMARY KEY (product_id, promotion_id),
  CONSTRAINT fk_pp_product        FOREIGN KEY (product_id)
    REFERENCES product (product_id)
    ON UPDATE CASCADE
    ON DELETE CASCADE,
  CONSTRAINT fk_pp_promotion      FOREIGN KEY (promotion_id)
    REFERENCES promotion (promotion_id)
    ON UPDATE CASCADE
    ON DELETE CASCADE
);

-- =====
-- END OF DDL
-- =====

```

ANALYSIS: i got an error and inserted the error into my chat with [CLAUDE.AI](#)

Prompt: this returned: ERROR 1044 (42000) at line 7: Access denied for user 'user_44p2rjcp5_44p4qg2c2'@'%' to database 'jaymart'

Output: That error means the database user doesn't have permission to run **CREATE DATABASE** — which is common in shared or managed MySQL environments where the database is pre-created for you.

The fix is simple: remove the **CREATE DATABASE** and **USE** statements and run the DDL directly against your existing database. The two removed lines were:

```

CREATE DATABASE IF NOT EXISTS jaymart ...;
USE jaymart;

```

When you run the script now, connect to your target database first — either by selecting it in your tool (e.g. phpMyAdmin, MySQL Workbench, TablePlus) or by specifying it in your CLI command like so: `mysql -u your_user -p your_database_name < jaymart_ddl.sql` Everything else in the script remains identical — all tables, keys, and constraints are unchanged.

ANALYSIS: I copied the code it gave me and this displayed nothing which is good, but I ran 'SHOW TABLES;' just to make sure it was good to go for the next part. Down below is what displayed:

```
Output:
+-----+
| Tables_in_sandbox_db |
+-----+
| attendance            |
| category              |
| customer              |
| department            |
| employee              |
| inventory              |
| payment               |
| product               |
| product_promotion     |
| promotion             |
| supplier              |
| transaction            |
| transaction_line      |
+-----+
```

3b.

AI generated tests:

TEST 1. INSERT

```
INSERT INTO customer (first_name, last_name, email, phone, loyalty_tier, registered_at)
VALUES ('John', 'Doe', 'john.doe@email.com', '555-111-2222', 'silver', '2026-01-01');
```

PASS: displayed nothing

TEST 2. INSERT

```
INSERT INTO customer (customer_id, first_name, last_name, email, loyalty_tier, registered_at)
VALUES (1, 'Jane', 'Smith', 'jane.smith@email.com', 'gold', '2026-01-02');
```

FAIL: error because duplicate key '1' is already assigned

TEST 3. UPDATE

```
UPDATE customer
```

```
SET loyalty_tier = 'gold'
```

```
WHERE customer_id = 1;
```

PASS: displayed nothing.

TEST 4. UPDATE

```
UPDATE inventory
```

```
SET quantity_on_hand = -10
```

```
WHERE product_id = 1;
```

FAIL: The UPDATE statement did not raise an error even though it violates the CHECK constraint.

TEST 5. DELETE

DELETE FROM customer WHERE customer_id = 1;

PASS: displayed nothing

Student generated:

INSERT INTO product (name, price, unit_of_measure, category_id, supplier_id)
VALUES ('Gaming Console', -300, 'unit', 1, 1);

Expected outcome was fail

ERROR 3819 (HY000) at line 267: Check constraint 'ck_product_price' is violated.

I wanted to try this constraint because the ai generated test did not raise an error before

INSERT INTO customer (first_name, last_name, email, phone, loyalty_tier, registered_at)
VALUES ('Olivia', 'Brown', 'olivia.brown@email.com', '555-222-3333', 'standard', '2026-01-10');

Expected outcome was pass and did

INSERT INTO customer (customer_id, first_name, last_name, email, phone, loyalty_tier,
registered_at)

VALUES (100, 'Maya', 'Rivera', 'maya.rivera@email.com', '555-123-4567', 'standard',
'2026-01-10');

INSERT INTO customer (customer_id, first_name, last_name, email, phone, loyalty_tier,
registered_at)

VALUES (100, 'Duplicate', 'Customer', 'duplicate.customer@email.com', '555-999-8888', 'silver',
'2026-01-11');

Expected outcome was fail because second INSERT violates primary key constraint

ERROR 1062 (23000) at line 270: Duplicate entry '100' for key 'customer.PRIMARY'

INSERT INTO transaction (customer_id, employee_id, total_amount)

VALUES (999, 999, 50.00);

Expected outcome was fail, customer_id and employee_id don't exist in referenced tables which violates FK constraints

ERROR 1452 (23000) at line 267: Cannot add or update a child row: a foreign key constraint fails (`sandbox\_db`.`transaction`, CONSTRAINT `fk\_txn\_customer` FOREIGN KEY (`customer\_id`) REFERENCES `customer` (`customer\_id`) ON DELETE RESTRICT ON UPDATE CASCADE)

INSERT INTO employee (first_name, last_name, email, role, salary, hire_date, department_id)
VALUES ('James', NULL, 'james@email.com', 'Cashier', 18.00, '2026-01-01', 1);

Expected outcome was fail because last_name can't be null

ERROR 1048 (23000) at line 267: Column 'last_name' cannot be null

Multi-table JOIN queries weren't displaying anything so i added sample data:

```
INSERT INTO supplier (supplier_name, contact_name, email, phone)
VALUES ('Best Tech Supplier', 'Sam Lee', 'sam@besttech.com', '555-1111');

INSERT INTO department (name)
VALUES ('Electronics');

INSERT INTO employee (first_name, last_name, email, role, salary, hire_date, departm
VALUES ('Alex', 'Smith', 'alex@jaymart.com', 'Cashier', 40000.00, '2026-01-01', 1);

INSERT INTO category (name, department_id)
VALUES ('Phones', 1);

INSERT INTO product (name, price, unit_of_measure, category_id, supplier_id)
VALUES ('iPhone Charger', 19.99, 'unit', 1, 1);

INSERT INTO customer (first_name, last_name, email, phone, loyalty_tier, registered_
VALUES ('Maria', 'Lopez', 'maria@email.com', '555-2222', 'standard', '2026-01-02');

INSERT INTO transaction (customer_id, employee_id, total_amount)
VALUES (1, 1, 19.99);

INSERT INTO transaction_line (transaction_id, product_id, quantity, unit_price)
VALUES (1, 1, 1, 19.99);

SELECT
c.first_name AS customer_first_name,
c.last_name AS customer_last_name,
e.first_name AS employee_first_name,
e.last_name AS employee_last_name,
p.name AS product_name,
tl.quantity,
tl.unit_price,
t.total_amount
FROM transaction t
JOIN customer c
ON t.customer_id = c.customer_id
JOIN employee e
ON t.employee_id = e.employee_id
JOIN transaction_line tl
ON t.transaction_id = tl.transaction_id
JOIN product p
ON tl.product_id = p.product_id;
```

Output:

customer_first_name	customer_last_name	employee_first_name	employee_last_name	product_name	quantity	unit_price	total_amount
Maria	Lopez	Alex	Smith	iPhone Charger	1	19.99	19.99

My student-designed test suite better targeted specific failure points because each test was chosen to match a required database constraint. These tests produced clear results and error messages, which made it easier to confirm that the schema was enforcing important rules correctly. Nonetheless, the AI-generated test suite was more comprehensive because it tested more areas of database behavior, including valid inserts, duplicate primary keys, updates to existing records, CHECK constraints, and delete behavior. However one of the AI-generated tests was not as precise because it expected a CHECK constraint failure, but the UPDATE statement didn't actually raise an error.

3c.

Example of data population

```
413 INSERT INTO transaction_line VALUES
414 (4, 4, 4, 1, 6.99),
415 (5, 5, 5, 1, 3.49),
416 (6, 6, 6, 1, 18.99),
417 (7, 7, 7, 1, 9.99),
418 (8, 8, 8, 1, 11.99),
419 (9, 9, 9, 1, 22.99),
420 (10, 10, 10, 1, 24.99);
421
422 INSERT INTO payment VALUES
423 (4, 4, 'cash', 6.99),
424 (5, 5, 'card', 3.49),
425 (6, 6, 'mobile', 18.99),
426 (7, 7, 'card', 9.99),
427 (8, 8, 'voucher', 11.99),
428 (9, 9, 'card', 22.99),
429 (10, 10, 'cash', 24.99);
430
431 INSERT INTO promotion VALUES
432 (4, 'Home Goods Weekend Deal', 0.1000, '2026-01-10', '2026-01-20'),
433 (5, 'Dairy Discount Day', 0.0500, '2026-01-10', '2026-01-12'),
434 (6, 'Toy Tuesday Sale', 0.1500, '2026-01-11', '2026-01-20'),
435 (7, 'Book Club Savings', 0.1000, '2026-01-10', '2026-01-31'),
436 (8, 'Cleaning Essentials Deal', 0.1200, '2026-01-11', '2026-01-25'),
437 (9, 'Fitness Starter Sale', 0.1800, '2026-01-15', '2026-02-01'),
438 (10, 'Pet Care Promo', 0.0800, '2026-01-10', '2026-01-30');
439
440 INSERT INTO product_promotion VALUES
441 (4, 4),
442 (5, 5),
443 (6, 6),
444 (7, 7),
445 (8, 8),
446 (9, 9),
447 (10, 10);
448
449 SELECT * FROM supplier;
450 SELECT * FROM department;
451 SELECT * FROM employee;
452 SELECT * FROM category;
453 SELECT * FROM product;
454 SELECT * FROM inventory;
455 SELECT * FROM customer;
456 SELECT * FROM attendance;
457 SELECT * FROM 'transaction';
458 SELECT * FROM transaction_line;
459 SELECT * FROM payment;
460 SELECT * FROM promotion;
461 SELECT * FROM product_promotion;
```

STDIN

Input for the program (Optional)

employee_id	first_name	last_name	email	role	salary	hire_date	department_id
1	Alex	Smith	alex@jaymart.com	Cashier	40000.00	2026-01-01	1
2	Nina	Brown	nina@jaymart.com	Stock Associate	38000.00	2026-01-03	2
3	Chris	Miller	chris@jaymart.com	Sales Associate	39000.00	2026-01-05	3
4	Taylor	Green	taylor@jaymart.com	Cashier	41000.00	2026-01-06	4
5	Jordan	Lee	jordan@jaymart.com	Sales Associate	39000.00	2026-01-07	5
6	Riley	Adams	riley@jaymart.com	Stock Associate	37000.00	2026-01-08	6
7	Casey	Wilson	casey@jaymart.com	Cashier	40500.00	2026-01-09	7
8	Morgan	Clark	morgan@jaymart.com	Sales Associate	39500.00	2026-01-10	8
9	Jamie	Lewis	jamie@jaymart.com	Stock Associate	37500.00	2026-01-11	9
10	Avery	Walker	avery@jaymart.com	Manager	60000.00	2026-01-12	10

category_id	name	department_id
6	Board Games	6
10	Dog Food	10
7	Fiction	7
9	Fitness	9
4	Kitchen	4
8	Laundry	8
5	Milk	5
1	Phones	1
3	Shirts	3
2	Snacks	2

product_id	name	price	unit_of_measure	category_id	supplier_id
1	iPhone Charger	19.99	unit	1	1
2	Organic Apples	4.99	bag	2	2
3	Jay-Mart T-Shirt	12.99	unit	3	3
4	Coffee Mug	6.99	unit	4	4
5	Whole Milk	3.49	gallon	5	5
6	Family Board Game	18.99	unit	6	6
7	Mystery Novel	9.99	unit	7	7

PART IV

Reflection and critical analysis

Using GenAI throughout this database project was helpful, but it also showed me that AI output can't be accepted without reviewing. Across all three parts of the project, the biggest inaccuracies usually came from either overcomplication or lack of precision. In Part 1, the AI sometimes generated too many entities and made the database feel more complicated than necessary for a small department store. It included useful ideas, but some were closer to an enterprise retail system than a beginner-level local store database. In Part 2, the ER diagram output was sometimes difficult to understand because the first version used Crow's Foot notation, which would be hard for a student without database training to interpret. In Part 3, the AI-generated code was mostly useful, but some tests did not behave as expected. My own manually designed tests ended up being more reliable because I intentionally targeted specific constraints one by one. Overall, the AI was strongest at generating structure and examples, but weaker when exact correctness, execution behavior were required.

My prompting strategy changed a lot as the project went on. At first, my prompts were broad, so the AI responded broadly. That helped me brainstorm, but it also created extra complexity. As I got further into the project, I was more specific by asking for exact entities, clearly labeled pks and fks, simpler explanations, and readable diagrams. The most effective technique was giving the AI my exact requirements and telling it what format I wanted the answer in. For example, asking it to withhold the ER diagram and only list entities and relationships gave me a much cleaner response. Asking for Chen's notation also worked better because it made the ER diagram easier to read and understand. The least productive prompts were the ones that were too general or when I asked for a downloadable diagram.

This project also showed me that domain knowledge matters a lot when using AI for database design. I do not think a student with no prior database training could successfully complete this project using AI alone, at least not with full confidence. AI can produce something that looks polished, but the student still has to know what to check. I also had to recognize that CREATE TABLE and INSERT statements might not display data unless I used SELECT after. AI was useful, but my own understanding was necessary to catch issues and ask better follow-up questions.

There were also challenges caused by switching AI conversations. I ran out of chat space with my first AI agent and had to continue with another one. Since the new AI session had no memory of the earlier discussions, I had to rebuild the project from the beginning by re-explaining the business rules, entities, preferred attributes, and relationships. During the restart, I noticed that the AI sometimes suggested slightly different entity names and relationship structures compared to the earlier outputs.

In a professional database project, there are some situations where AI-generated output could be trusted with minimal review when the ethical and business risks are low.

- **AI can quickly create fake customer, employee, or transaction records for testing purposes. Since the data is not real, there are fewer concerns caused by mistakes.**
- **AI is useful for creating first drafts. Even if the wording is imperfect, the consequences are usually minor.**

However, there are also situations where AI-generated output shouldn't be trusted without careful human overview because of moral and security concerns.

- **AI shouldn't be trusted with databases containing payroll records, customer payment information, etc without extensive review. A mistake in design or data handling could expose sensitive information and violate privacy laws.**
- **AI can silently damage data integrity. Relying too heavily on AI without validation could lead to financial loss or unfair outcomes for users.**